גרסה 3 - רכיב שמירת נתונים לאורך זמן, וחסינות מפני נפילות -2026

גרסה 3 של מערכת המסחר עוסקת בהעשרת המערכת ברכיב שמירת נתונים לאורך זמן (persistence). דרישות החסינות והתאוששות מחייבות את היכולת לשחזר את מצב המערכת במקרה של הפעלה יזומה מחדש וכמו כן במקרים לא רצויים כגון נפילת המערכת, איבוד תקשורת וכו'. לשם כך יש לחבר בסיס נתונים למערכת. החיבור ייעשה באמצעות רכיב מקשר בין המערכת לבסיס הנתונים באמצעות ספריית **ORM** לבחירתכם (למשל Spring ב-Java).

הדרישות לגרסה 3:

1. דרישות רמת שירות:

a. דרישה 7 – שמירה לאורך זמן:
המערכת שומרת את מצבה העדכני **במסד חיצוני של נתונים**, באופן המאפשר הפרדה בין פעילות חישובית לנתונים. ההפרדה מאפשרת שחזור מצב מערכת וניהול יעיל.
יש לתמוך במסד נתונים מרוחק (לא באותו מחשב עליו רצה המערכת).

b. דרישה 5 – חסינות מפני נפילת המערכת בתרחישים לא רצויים:
   i. המערכת צריכה להתמודד עם אובדן תקשורת בין רכיבים, ועם מערכות חיצוניות.
   ii. המערכת צריכה להיות **חסינה** להתנהגויות לא צפויות של מערכות חיצוניות, כגון: כישלון בשליחת בקשה, ממשק לא תואם, אי מענה לבקשה וכו'.

c. דרישה 6 – התאוששות מתקלות: המערכת צריכה לאפשר חזרה לפעילות רגילה לאחר תקלה או מקרה קיצון. למשל, באם ניתק הקשר עם מסד הנתונים, המערכת צריכה לחזור לפעול כרגיל באופן אוטומטי לאחר חידוש הקשר.

2. דרישות פונקציונאליות:

a. עדכון דרישה **I**.1 – אתחול המערכת:
   i. אתחול המערכת מתבצע על פי קובץ (או מס' קבצי) קונפיגורציה חיצוני (בפורמט לבחירתכם) המגדיר את הפרמטרים לאתחול. למשל, הקבצים צריכים להגדיר כמה משתמשים יכולים לבצע שמירת כרטיסים עבור אירוע בו זמנית לפני שנדרשים לעמוד בתור. כמו כן, פרטי החיבור למסד נתונים צריכים להופיע באחד מקבצי הקונפיגורציה ולא להיות hard coded במערכת.
   ii. אפשרות אתחול במצב (state) המוגדר בקובץ חיצוני (קובץ נוסף, נפרד מקובץ הקונפיגורציה): לאחר אתחול המערכת על פי קובץ הקונפיגורציה (i), והפעלת הפעולות בקובץ מצב-התחלתי, המערכת נמצאת במצב המוגדר בקובץ זה. קובץ המצב-התחלתי מכיל הנחיות לסדרה של סיפורי שימוש וארגומנטים להפעלתם (פורמט כלשהו). בשלב האתחול, המערכת תקרא את הקובץ ותפעיל את סיפורי השימוש המתוארים בו, כך שבסיום האתחול המערכת תהיה במצב הנדרש.

b. דרישות **I.3** ו-**I.4** – חיבור למערכת תשלום והנפקת כרטיסים. על המערכת להתקשר עם מערכות התשלום והנפקת הכרטיסים החיצוניות המפורטות במסמך הנלווה.

דגשים למימוש:

1. שמירת נתונים לאורך זמן: יש להשתמש בספריית ORM כלשהי על מנת לקשר בין המערכת לבסיס הנתונים. מפורט בנספח למסמך זה.
2. אתחול מקובץ מצב-התחלתי: הסבר ודוגמה מופיעים בנספח למסמך זה.
3. הסבר לקבצי אתחול: דף הפרויקט ב-GitHub צריך להכיל קובץ **readme**, המסביר כיצד לאתחל את המערכת ומגדיר את הפורמטים של קובץ הקונפיגורציה ושל הקובץ מצב-התחלתי.
4. בדיקות:

a. יש לכתוב בדיקות עבור חסינות המערכת בהתמודדות עם המערכות החיצוניות ומסד הנתונים.

b. יש לכתוב בדיקות עבור אתחול תקין: המערכת לא עולה אם האתחול אינו תקין.

c. יש להשתמש **בקובץ קונפיגורציה ייעודי** לצורך הבדיקות בכדי שהבדיקות לא יפנו למערכות החיצוניות האמיתיות ועל מנת שלא "יזהמו" את מסד הנתונים המשמש את המערכת.

5. יש לעדכן את מודל המחלקות, יש לעדכן את תרחישי השימוש עבור אתחול המערכת.
6. הארכיטקטורה העדכנית של המערכת תכלול את הרכיבים הבאים:
   - רכיב לקוח
   - רכיב תקשורת
   - רכיב אפליקציה (**application**)
   - רכיב תשתית (**infrastructre**)
   - רכיב **domain**
   - ה-**ORM** מהווה את את **Data Access Layer**, לכן לא צריך לצייר שכבה כזו מפורשות
   - מסד נתונים

תוצרי גרסה 3

1. מודלים עדכניים: תרחישי שימוש, ארכיטקטורה ומחלקות.
2. מימוש.
3. בדיקות: עלפי הדרישות החדשות. עבור כל הסוגים. לא יתקבל מימוש ללא רכיב בדיקות משמעותי!

נספח לגרסה 3

שמירה לאורך זמן

1. ספריית ה-**ORM** מהווה **Data Access Layer** המפרידה בין לב-המערכת לבסיס-הנתונים. ספריות אלו מאפשרות הגדרה של התאמה בין ישויות לב-המערכת לישויות בסיס-הנתונים, ומטפלות בשמירה ואחזור של מידע ב/מ-בסיס הנתונים. הקישור בין לב-המערכת למסד-הנתונים הוא שכבתי: מכיוון המערכת אל הנתונים ולא להיפך. כלומר, יש לתכנן ולהתאים את השמות בספריית ה-ORM למימוש הקיים. **לא יתקבל פתרון המבוסס על תכנון עצמאי של מסד נתונים.**

2. יש להקפיד לא לשמור במסד הנתונים ערכים ואובייקטים שהם ארעיים במהותם, לדוגמה, תורי המתנה.

3. **שמירת נתונים בזיכרון** (caching) יש להשתמש במנגנון ה-cache המסופק כחלק מספריות ה-ORM.

4. טרנזקציות: יש להגדיר כל תרחיש שימוש כטרנזקציה נפרדת.

5. לצורך הפרויקט אתם יכולים (אך לא חייבים) להשתמש ב-Google Cloud Platform עם קרדיטים בהיקף של 50$. מכיוון שמדובר בפרויקט לימודי עם עומס נמוך יחסית, אין צורך בארכיטקטורה מורכבת או במשאבים חזקים. מומלץ להשתמש במכונה הווירטואלית הזולה ביותר לצורכי הפרויקט, ולהקצות נפח אחסון מינימלי. על מנת לגרום לקרדטים להספיק, יש להימנע משימוש בשירותים יקרים כגון GPUs Kubernetes , או מספר גדול של מכונות. קונפיגורציה מומלצת:

   Database Engine: PostgreSQL (or MySQL).
   Instance Type: db-f1-micro (Shared Core, 0.6 GB RAM). This is the cheapest option.
   Storage Type: HDD (Hard Disk Drive), instead of SSD for lower costs.
   Storage Size: Minimum (i.e., 10 GB).
   Region: Select a cost-effective region like us-east1 (South Carolina) or us-west1 (Oregon).
   Backups & HA: Disable Automated Backups and Disable High Availability (HA).

   בנוסף, **חשוב לכבות משאבים כאשר אינם בשימוש.** בפרט, ניתן לפתח את המערכת ע"י שימוש in-memory database מקומי (כמו H2) ולעבור להשתמש בשירותי הענן רק שהמערכת יציבה וכאשר מעוניינים לעשות deployment; שינוי כזה מצריך שינוי בקובץ הקונפיגורציה בלבד ואינו משנה את הקוד של המערכת.
   מומלץ גם להגדיר תקציב והתראות במערכת החיוב של Google Cloud, כדי לעקוב אחר ניצול הקרדיטים ולהימנע מחריגות.

אתחול מקובץ "מצב-התחלתי"

אתחול המערכת מתבצע מול שכבת האפליקציה וניתן לבצע באמצעותו רק פעולות חוקיות. למשל, לא ניתן לפתוח חברת הפקה מבלי לבצע חיבור למערכת. האתחול מאפשר את כל תרחישי השימוש של הדרישות הפונקציונליות.

למשל דוגמה אפשרית עבור פתיחת חברת הפקה והגדרת מנהל:

```
guest-registration(moshe,*moshe's password*,*other required information*);
guest-registration(rina,*rina's password*, *other required information*);
login(rina, *password*);
open-production-company(rina's token, company_name, *other required
          information*);
appoint-manager(rina's token, company_name, moshe, *other required
          information*);
```

תהליך האיתחול ממצב-התחלתי מסתיים בהצלחה רק אם כל הפעולות חוקיות ומסתיימות בהצלחה. אם אחת מהפעולות נכשלת על תהליך האתחול מקובץ מצב-התחלתי להיכשל ולדווח שגיאה בהתאם.

הקלות למשרתי מילואים:

הקלות לקבוצות עם היקף שירות מילואים משמעותי.
שירות המילואים בגרסה זו נקבע ע"פ מס' הימים המצטבר שחברי הקבוצה ביצעו **החל ממועד פרסום העבודה (24/5)**. לצורך גרסה זו, יספרו הימים **עד לתאריך 25/6 כולל**. דוגמה: נניח שחבר קבוצה אחד שירת 10 ימי מילואים בטווח התאריכים, חברת קבוצה שניה שירתה 20, ויתר חברי הקבוצה לא שירתו, סך ימי המילואים הקבוצתי הינו 30.

ההקלות הינם בהתאם לשלוש מדרגות:

1. מדרגה א': סך מצטבר של 21+ ימים (או 0+ ימים בקבוצות של 7 סטודנטים).
2. מדרגה ב': סך מצטבר של 42+ ימים (או 21+ ימים בקבוצות של 7 סטודנטים).
3. מדרגה ג': סך מצטבר של 63+ ימים (או 42+ ימים בקבוצות של 7 סטודנטים).

הקלות למשרתים לפי מדרגות:

קבוצות **במדרגה א'** אינן נדרשות לממש את הדרישות הבאות:

- אפשרות אתחול במצב (state) המוגדר בקובץ חיצוני

קבוצות **במדרגה ב'** אינן נדרשות לממש את הדרישות הבאות:

- כל הדרישות של מדרגה א'
- אתחול המערכת מתבצע על פי קובץ (או מס' קבצי) קונפיגורציה
- בדיקות (טסטים) עבור חסינות
- חיבור מסד הנתונים לשרת מרוחק

קבוצות **במדרגה ג'** אינן נדרשות לממש את הדרישות הבאות:

- כל הדרישות של מדרגה א' ומדרגה ב'
- חיבור המערכת למערכות חיצוניות